

Day 7 (27/01/26)

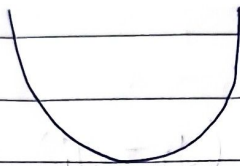
• Lab (Gradient Descent + N.N from scratch)

Loss = calculated over forward pass

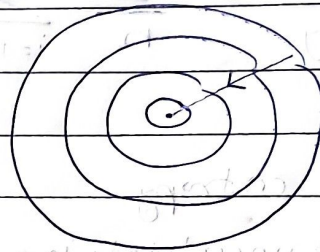
Loss function criteria: (i) monotonic (P)

(ii) differentiable

• Note in 3D:



Contour graph/plot:
(top view)



take gradient
and go to
that minimum
point

gradient - tells direction
learning rate - how much
Step

~~How big can a
Confusion matrix
be? (depends on
no. of class
labels)~~

W74

classmate

Date
Page

Day 7 (27/01/26)

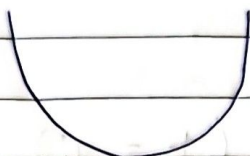
• Lab (Gradient Descent + N.N from scratch)

Loss - calculated over forward pass

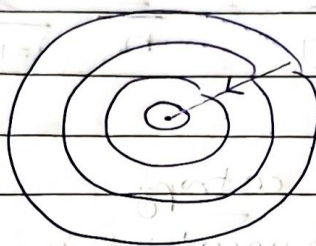
Loss function criteria: (i) monotonic (P)

(ii) differentiable

• Note in 3D:



Contour graph/plot:
(top view)



take gradient
and go to
that minimum
point

gradient - tells direction

learning rate - how much

Step

Day 8 (28/01/26)

• Use PyTorch instead of

tensorflow

PyTorch

Lab (NN. with python master)

• Tensor - (vector of n dimensions)

↳ Specifically called tensor

in n.n

↳ fundamental unit of PyTorch lib

• NN req GPU (if cuda available $\Rightarrow$ GPU present)

$\Rightarrow$ a library by Nvidia

• Datatypes generally preferred = float

• 3 properties of tensor = shape

• rand $\leftarrow$ std deviation

randn $\leftarrow$ normal deviation } $\Rightarrow$ may lead to

blow up of G.D

how big can a
Confusion matrix

be? (depends on
no. of class
labels)

- $b = a$ in tensor $\equiv$ address copying
- $b = \text{clone}(a)$ $\equiv$ Copying the element
- squeeze and unsqueeze
 - $\uparrow$ reduce dimension
 - $\uparrow$ increase the dimension
- $\text{view}()$ compatible with contiguous tensor only
- Masking of tensor $\Rightarrow$ to zero out or change the value of tensors !!
 - $\Rightarrow$ req mask of same size as tensor
- Structure + flow : imp. for N.N
- grad-fn : tells whether ^{weights can be} updated
 - $\Rightarrow$ true : for training
 - false : while testing

- $b = a$ in tensor $\equiv$ address copying
- $b = \text{clone}(a) \equiv$ Copying the element
- Squeeze and unsqueeze
 - $\uparrow$ reduce dimension $\uparrow$ increase the dimension
- $\text{view}()$ Compatible with contiguous tensor only
- Masking of tensor $\Rightarrow$ to zero out or change the value of tensors !!
 - $\Rightarrow$ Req mask of same size as tensor
- Structure + flow : imp. for n-n
- grad-fn : tells whether ^{weights can be} updated
 - $\Rightarrow$ true : for training
 - false : while testing

Day 9 (03/02/26)

Error Functions (for supervised model)

- (i) MSE : Mean Square Error
- (ii) Cross Entropy

Bayes Theorem

$$P(M|D) = \frac{P(D|M) \cdot P(M)}{P(D)}$$

$$\log(P(M|D)) = \log P(D|M) + \log(P(M))$$

$\nearrow$ Equally likely
so not considered

$$- \log(P(D))$$

$\hookrightarrow$ as not depended on M

$$\Rightarrow \log(P(M|D)) \approx \log P(D|M)$$

$\nearrow$ $\nearrow$ MAPE

$\hookrightarrow$ Maximizing A Priori Error

$\nearrow$ MLE : Maximizing Likelihood Estimate

Maximizing this $\equiv$ minimizing error ∇

$$E = -\log(P(M|D)) = -\log(P(D|M))$$

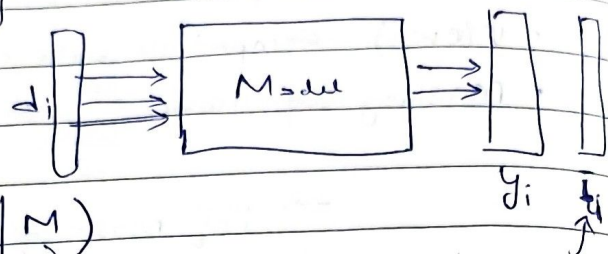
$$w' = w_0 + \frac{\partial E}{\partial w}$$

collection of samples

Error : $E = -\log P(D|M)$ ← objective function
⇒ Minimising E

model
(as in weight param)

$$D = \{ (d_1, t_1), (d_2, t_2), \dots, (d_n, t_n) \}$$



$$E = -\log P((d_1, t_1), (d_2, t_2), \dots, (d_n, t_n) | M)$$

as model is weight param (ground truth)

$$= -\log \prod_{i=1}^n P(\langle d_i, t_i \rangle | w)$$

$$= -\log \prod_{i=1}^n P(t_i | d_i, w) \cdot P(d_i | w)$$

$$= -\log \prod_{i=1}^n P(t_i | d_i, w) \cdot P(d_i) \leftarrow \text{as not dependent on model parameters}$$

(hence not optimised)

NOTE : Used in unsupervised models

Relative Entropy (KL Divergence)

$$D(P || Q) = -\sum P \log(P/Q)$$

Truth distribution Predicted distribution

Cross Entropy ←

$$CE(P, Q) = H(P) + D(P || Q)$$

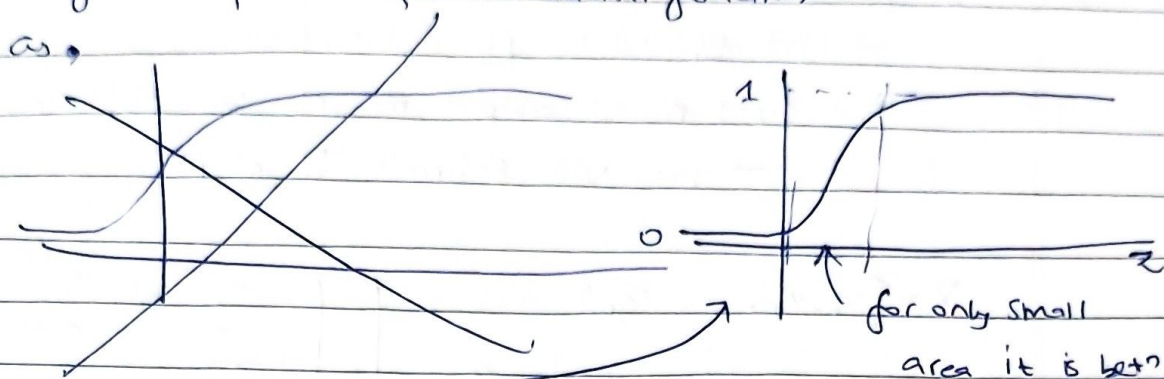
how much bits of info. req. to represent P by Q Actual entropy divergence

$$= -\sum P \log Q$$

$$= -\log \prod P_i Q_i = -\log \prod P_i - \log \prod Q_i = H(P) + D(P || Q)$$

(I) Binary Classifier

- uses sigmoid function for activation function



$$f(z) = \frac{1}{1 + e^{-z}}$$

(therefore it can be treated as probability also)

$\Rightarrow$ either it falls to 0 or 1

$$E = -P(t_i | x_i, w)$$

Assumption: If y_i is the probability of the $^1(t_i = 1)$ then,

$(1 - y_i)$ is the probability of -ve class

$$P(t_i | x_i, w) = \begin{cases} y_i & \text{if } t_i = 1 \\ 1 - y_i & \text{if } t_i = 0 \end{cases}$$

using this

$$P(t_i | d_i, w) = y_i^{t_i} (1 - y_i)^{(1-t_i)} \quad \leftarrow \text{Binomial distribution}$$

$$\log(P(t_i | d_i, w)) = \underbrace{t_i \log y_i}_{\text{true}} + \underbrace{(1 - t_i) \log(1 - y_i)}_{\text{predicted}}$$

$\nwarrow$ this is CE !! (or binary

$$\text{if } t_i = 0 \Rightarrow \log(P(t_i | d_i, w)) = \log(1 - y_i)$$

CE)

$\uparrow$
as 2

$$\text{if } t_i = 1 \Rightarrow \log(P(t_i | d_i, w)) = \log(y_i)$$

Classes

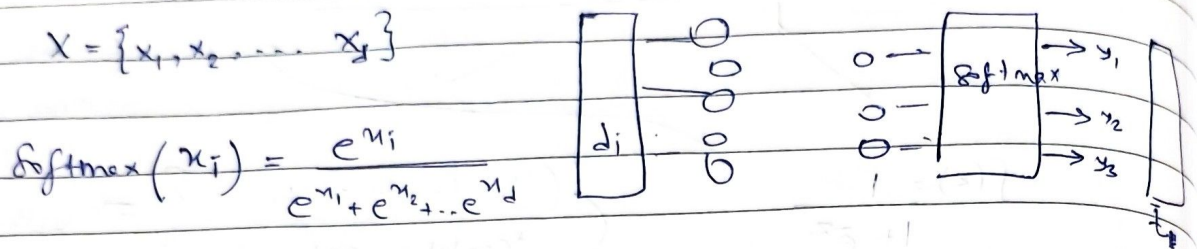
(ii) Multiclass Classifier

↓

we req t_i to be pb distribution

but direct model output may not be pb distribution

→ we use softmax to get pb



→ Multinomial distribution

$$P(t_i | d_i, w) = y_1^{t_{i1}} y_2^{t_{i2}} \dots y_d^{t_{id}}$$

$$= \prod_{i=1}^d y_i^{t_i}$$

$$\log(P(t_i | d_i, w)) = \sum_{i=1}^d \underset{\substack{\uparrow \\ \text{true} \\ \text{distribution}}}{t_i} \log \underset{\substack{\uparrow \\ \text{predicted} \\ \text{distribution}}}{y_i}$$

$$E = - \sum_{i=1}^d t_i \log y_i \quad (\text{Cross Entropy})$$

REGRESSION

dataset

$$f(\boxed{}, x_{\text{new}}) = \text{Confidence}$$

If linear line $\Rightarrow$ linear regressionIf non " " $\Rightarrow$ nonlinear " "

① Linear Regression

$$y = mx + c$$

Objective: to find m & c which minimises the error

$$E = \sum (t_i - y_i)$$

Since it can be +ve or -ve $\Rightarrow$ cancellation

options: (i) $\sum |t_i - y_i|$

(ii) $\sum ||t_i - y_i||^2 \leftarrow \text{MSE}$

$$E = \sum_{i=1}^N ||t_i - (mx_i + c)||^2 \leftarrow \text{Sum Square error}$$

Day 10 (09/02/20)

$$(-1 \cdot (-1) = 0)$$

$$\frac{\partial E}{\partial c} = 2 \sum_{i=1}^N (y_i - mx_i - c) = 0$$

$$+1 \cdot (-1) = -1$$

$$\sum_{i=1}^N y_i - m \sum_{i=1}^N x_i - Nc = 0$$

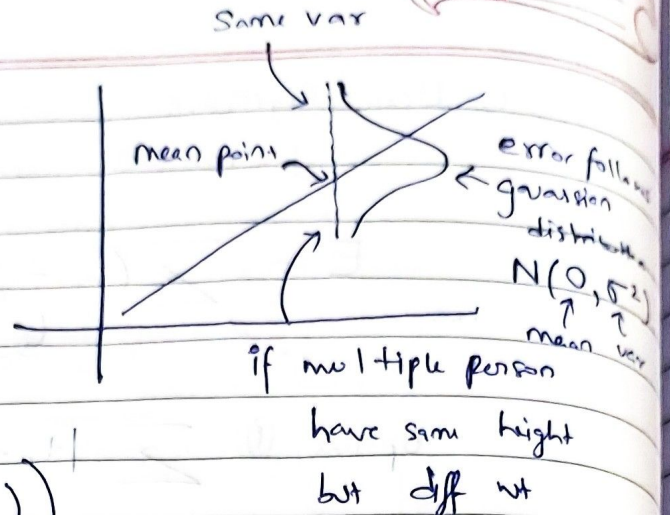
$$Nc = + \sum_{i=1}^N y_i - m \sum_{i=1}^N x_i$$

$$c = + \bar{y} - m \bar{x}$$

$$\frac{\partial E}{\partial m} = 2 \sum_{i=1}^N x_i (y_i - mx_i - c) = 0$$

$$m = \frac{\sum_{i=1}^N (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^N (x_i - \bar{x})^2} = \frac{\text{Cov}(x, y)}{\text{Var}(x)}$$

- Deriving the error now from likelihood estimate to prove the error fn we got can be optimised



$$E = -\log(P(t_i | d_i, w))$$

Since gaussian distribution considered,

$$P(x) = \frac{1}{\sqrt{2\pi}\sigma^2} e^{-\frac{(x-\bar{x})^2}{2\sigma^2}}$$

for our case,

$$P(t_i) = \frac{1}{\sqrt{2\pi}\sigma^2} e^{-\frac{(t_i - \hat{y}_i)^2}{2\sigma^2}}$$

predicted ($\hat{y}$) mean point

$$E = -\log\left(\frac{1}{\sqrt{2\pi}\sigma^2} e^{-\frac{(t_i - \hat{y}_i)^2}{2\sigma^2}}\right)$$

$$E = -\log\left(\frac{1}{\sqrt{2\pi}\sigma^2}\right) + \frac{(t_i - \hat{y}_i)^2}{2\sigma^2}$$

$$= \log \sqrt{2\pi} + \log \sigma^2 + \frac{(t_i - \hat{y}_i)^2}{2\sigma^2}$$

Since

Constant

here σ assumed as 1

$$= \frac{(t_i - \hat{y}_i)^2}{2} \leftarrow \text{error for single point}$$

$$(\bar{x} - i\bar{x}) (\bar{x} - i\bar{x}) \sum_{i=1}^n = n$$

$$\sum_{i=1}^n (\bar{x} - i\bar{x})$$

$$E = -\log (P(T|D, w))$$

$$= -\log \prod_{i=1}^n P(t_i | D_i, w) \quad \rightarrow \text{as each data point independent}$$

$$= \sum_{i=1}^n \frac{(t_i - \bar{y}_i)^2}{2} \quad \leftarrow \text{(can be ignored)}$$

$$\approx \sum_{i=1}^n (t_i - \bar{y}_i)^2$$

#univariate

Q What about regression (any)? (not just linear)

$$y_i = b_0 + b_1 x_i + b_2 x_i^2 + b_3 x_i^3 + \dots + b_k x_i^k$$

$$\hat{y}_1 = b_0 + b_1 x_1 + b_2 x_1^2 + \dots + b_k x_1^k + e_1$$

$$\hat{y}_2 = b_0 + b_1 x_2 + b_2 x_2^2 + \dots + b_k x_2^k + e_2$$

$$\vdots$$

$$\hat{y}_n = b_0 + b_1 x_n + b_2 x_n^2 + \dots + b_k x_n^k + e_n$$

$$\begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \hat{y}_3 \\ \vdots \\ \hat{y}_n \end{bmatrix} = \begin{bmatrix} 1 & x_1 & x_1^2 & \dots & x_1^k \\ 1 & x_2 & x_2^2 & \dots & x_2^k \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^k \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ \vdots \\ b_k \end{bmatrix} + \begin{bmatrix} e_1 \\ e_2 \\ \vdots \\ e_n \end{bmatrix}$$

$$\hat{\mathbf{y}} = \mathbf{A} \mathbf{b} + \mathbf{\bar{e}}$$

$\uparrow$ predicted $\uparrow$ ground truth $\uparrow$ error
 $\uparrow$ ground truth $\uparrow$ predicted

$$\Rightarrow E = (\hat{\mathbf{y}} - \mathbf{A} \mathbf{b})^2$$

• What if multivariate?

$$\begin{array}{c|c} x_1 & x_2 \end{array} \quad y$$

$$y_i = \underbrace{m_1 x_{1i} + m_2 x_{2i} + c}_{\hat{y}_i} + \epsilon_i$$

• if data is complete: we use MLE

else EM algorithm

↳ expectation maximization

→ fill the gap

→ apply MLE

→ update the gap

→ then again

} for parameter estimation

• What if multivariate?

$$\begin{array}{c|c} x_1 & x_2 \end{array} \quad y$$

$$y_i = \underbrace{m_1 x_{1i} + m_2 x_{2i} + c}_{\hat{y}_i} + \epsilon_i$$

• if data is complete: we use MLE

else EM algorithm

↑ expectation maximisation

} for parameter estimation

→ fill the gap

→ apply MLE

→ reupdate the gap

→ then again

Day 11 (03/02/26)

→ Chosen on basis of:

a) nature of the ps

(c) depth of

b) nature of target o/p

the network

• Activation Functions

→ applied over the linear weighted summation of the incoming info to a node



output: linear/non linear signal

→ it decides whether to activate a node

→ Features: (a) monotonic +

(b) differentiable

(c) quickly converging

$$-(J^T \bar{A} - \bar{b}) = \bar{b} \leftarrow$$

→ types : (i) linear
(ii) non-linear

linear

$$f(x) = ax + b$$

$$\frac{df(x)}{dx} = a \quad \leftarrow \text{constant gradient}$$

non-linear

a) Sigmoid (logistic)

{ as $f'(x)$ is function

of $f(x)$ }
and already
computed

$$f(x) = \frac{1}{e^{-x} + 1} \quad \frac{df(x)}{dx} = f(x)(1 - f(x))$$

• maps any real

input → between 0 and 1

• efficient in backprop
• but, vanishing

gradient pb exists

• S shaped curve: small $x \rightarrow \text{near } 0$

large $x \rightarrow \text{near } 1$

• not zero centered : as it is not symmetric
around 0

i.e.,



b) Tanh Activation function

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



$$f'(x) = 1 - (f(x))^2$$

⇒ at very large or small x

$$f(x) \approx 0 \text{ or } 1$$

$$f'(x) \approx 0$$

⇒ good for ~~output~~ layer
not for hidden layer

⇒ used in binary classification

(i) output $\in [-1, 1]$

(ii) Zero centered

(iii) can saturate or kill (vanish) gradients

↓

$$f'(x) = 1 - (f(x))^2 \approx 0 \text{ for large } x$$

⇒ gradients shrink

→ reaching to 0

(iv) gradient steeper than sigmoid ⇒ faster convergence!

$$\alpha e^x - \alpha$$

$$(\alpha e^x - \alpha + \alpha)$$

(c) Rectified Linear Unit (ReLU)

→ faster training speed

$$f(x) = \max(0, x)$$

→ reduces vanishing grad

$$f'(x) = 1$$

as $x > 0$ $f'(x) = 1$

(like vanishing gradient in one side) ← (but can be a pb) → $x < 0$ $f'(x) = 0$

gradient in

→ can blow up activation

one side)

→ dead nodes - if input always -ve

no wt update happens

(c1) Leaky ReLU

$$f(x) = \max(0.01x, x)$$

→ fixes dying ReLU

$$f'(x) = \begin{cases} 0.01, & x < 0 \\ 1, & x \geq 0 \end{cases}$$

(c2) Parametrized ReLU

$$f(x) = \max(\alpha x, x)$$

→ avoids dead neurons

$$f'(x) = \begin{cases} \alpha, & x < 0 \\ 1, & x \geq 0 \end{cases}$$

(d) Exponential Linear Unit (ELU)

$$f(x) = \begin{cases} \alpha (e^x - 1), & x < 0 \\ x, & x \geq 0 \end{cases}$$

• Can produce -ve o/p

$$f'(x) = \begin{cases} f(x) + \alpha, & x < 0 \\ 1, & x \geq 0 \end{cases}$$

• Can blow up activation function

Q What is vanishing gradient problem?

During backpropagation, gradients become extremely small (≈ 0) as they move backward through layers

As gradients (X) every layer
as in layer by layer

MLP

First step: representing sample in some vector format

i.e. vector $\rightarrow$ feature vector

feature decided by domain expert

Text Classification

- Collection of sentences (100 documents)
- Task: to classify the sentences to +ve/-ve (Sentiment)
- represent every sentence to a vector

- Possible features for representing text =

unigram	} for Bow
bigram	
trigram	
n-gram	

C — Collection

V — vocabulary $\{v_1, v_2, \dots\}$

- Representation Type: (i) Bow — # times the word present in doc (freq of word) is called term freq

(ii) TF-IDF

(iii) BM-25

- Idea: need a vector to give input to our network

- In case of BoW, since 't' no. of terms in the vocabulary may inc $\Rightarrow$ leading to sparsity
- $\Rightarrow$ It might also happen very similar words are present in vocab but a doc may not use

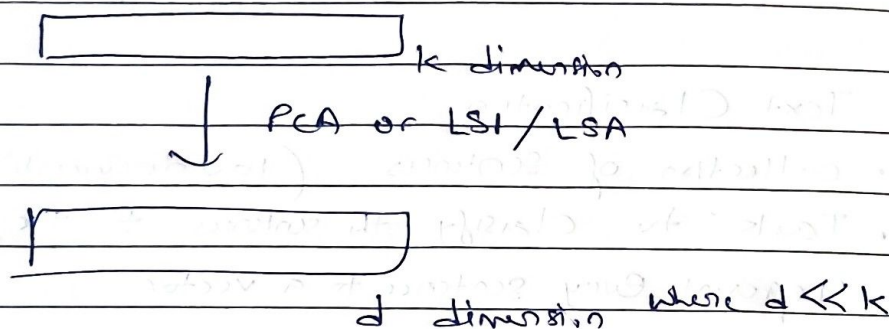


M1: hence we ~~use~~ use Embedding!

$\hookrightarrow$ hence we represent each word in a corpus

M2: reduce the dimension using PCA

i.e



NOTE: embedding $\equiv$ (i) are dense and low dimensional

(Similar words used in similar ways given similar nos)

- (ii) Capture semantics
- (iii) Solve BoW pbs directly

(i) Word 2 Vec

$|V| = K$ (size of vocabulary)

(Uses N.N to get & (i) Unique words find embeddings)

(ii) Input layers $\equiv$ no. of unique words

(Uses context of words)

(iii) hidden layer $\equiv$ how many ~~words~~ ^{nos} we want

$\nearrow$ (iv) be represented by (a word)

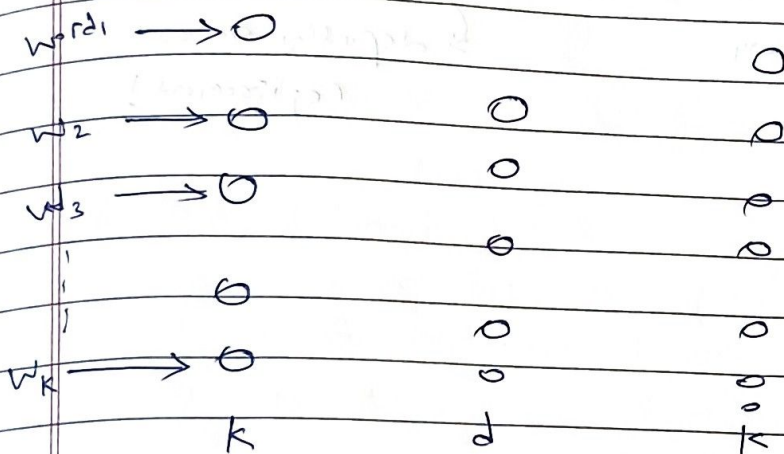
(iv) since this representation thing will be done in find way but the weights associated

With the neurons used are randomly initialised

⇒ training needs to be done (for that we do ^{context} ~~word~~ word prediction) $\rightarrow P(w_i/w_j)$

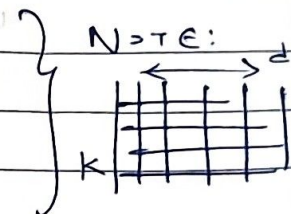
(v) So we add another layer $\equiv$ no. of unique words which when given a word gives us the next word

NOTE: input is one hot vector: i.e. 1 for the required word in vocab



Wts: $(k \times d)$ $(d \times k)$

$\{w_1, w_2, \dots, w_k\}$ $\equiv$ Set of words in Vocabulary



each row is an embedding of the word in the vocab!
input is one hot $\Rightarrow$ it selects one of the rows!

NOTE:

(i) in $k \times d \rightarrow$ each row represents embedding vector of one word ^{Size of vocab!}
~~each row gives input vector~~

NOTE:

here context and target defined

window size = 2 (eg)

$S_1: w_1, w_2, \textcircled{w_3}, w_4, w_5$
← target →

all words within window is

Context !!

can be found using $k \times d$ $\left\{ \begin{array}{l} \text{how similar it is to} \\ \text{current word} \end{array} \right.$

(ii) in $d \times k \rightarrow$ each column corresponds to a word $\Rightarrow$ it tells how strongly embedding features

Predict that word

$\Rightarrow$ it is not an embedding but output weights for prediction !!

- for each word, we define target and context pair

↑
look at both left and right of window size

- for above eg.

target	context
w ₁	w ₂
w ₁	w ₃
w ₂	w ₁
w ₂	w ₃
w ₂	w ₄

Target, context
one given as input
and other as
output
↓ depending on our
requirement!

represent similar words

classmate

Date _____
Page _____

- for each word, we define target and context pair

look at both left and right of window size

- for above eg.

target context

w₁ w₂

w₁ w₃

w₂ w₁

w₂ w₃

w₂ w₄

Target, context

one given as input

and other as

output

depending on our requirement!

Day 12 (11/02/26)

- Word2Vec

↳ 2 NN models

(i) CBOW - we do $Pr(\text{target} / \text{context})$

(ii) Skip gram - $Pr(\text{context} / \text{target})$

($>1 \times 1$)

how to give multiple context?

(i) set 1 for each one of them at the same time

(ii) give 1 at a time → then sum it up → (hidden layer)
to hidden layer

~~(channel)~~
multiple
input at
a time

(as sum will then do prediction
represent input but not all of the context)

→ low dimensional representation of a word

NOTE: embedding is the weight vector (word v)

- Glove etc - word2vec's extension

- After transformer - BERT is the state of the art

↳ used for embedding

gives contextual embedding

Dimensionality Reduction

→ good features: high dimension (feature selection)

as semantic connection may not be present

→ it refers to embedding itself — { PCA } initial embedding method

LSAA
matrix factorisation
reduction

for text

(~~is~~ not scalable)

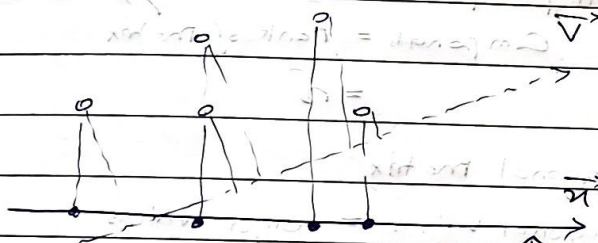
a) PCA

• a feature vector is projected on a line

→ we are trying to find a vector which can better represent our current features (vector)

It's all about projection!

ex -



→ instead of x, y
something else

Which is better $\vec{x}$ or $\vec{V}$? → $\vec{V}$ (Eigenvector)

→ (this is known as 1st Principal component)

Objective: find a vector which does maximal representation

• NOTE: Shifting origin to sample mean — in the process to find the

how to find if that vector is good?

vector

(The but in prev

⇒ Sum of Square error should

Plane before shifting will still remain

(be minimum (minimise))

(i) Instead of projection distance (modified Same)

(ii) take the distance from origin (deviation from

(maximise) Sample mean) — i.e. ~~variance~~ ~~std~~ deviation



NOTE: all projected points well separated

≡ In eigen vector (that's why good)

1st principal eigen vector — sum of sq
2nd principal component
distance
mean of sq
distance
orthogonal to each other

→ P.c feature vectors should be less correlated)

NOTE: Covariance reduction of feature vector

{ a good PC: (having maximum variance) } !!

Eigen Value decomposition

- if square matrix → eigen value decomposition

$$A = \begin{matrix} \text{eigenvector} \\ \text{1st and 2nd} \\ \begin{matrix} \boxed{U} \\ n \times n \end{matrix} \end{matrix} \begin{matrix} \begin{matrix} \boxed{\Sigma} \\ n \times n \end{matrix} \end{matrix} \begin{matrix} \begin{matrix} \boxed{U^{-1}} \\ n \times n \end{matrix} \end{matrix}$$

principal

Components = rank of matrix
= n

U values

- diagonal matrix

- diagonal values = eigen values

(decreasing order it is stored)

- If A symmetric and

orthogonal → U^{-1} will be U^T

as U will also be symmetric

out

- NOTE: we can't have K of ∞ non zero

and that represents A in reduced space

(denoted by U)

NOTE: how to find if a vector is good or not?

- minimise the Sum Square error } and the vector which
 → maximises the Sum Square distance } given these both properties
 ⇒ best fit vector!!

$$\bar{V} = d_1^2 + d_2^2 + d_3^2 + \dots + d_n^2$$

best fitting unit vector

(also called 1st Principal Component - Eigen vector)

AND, Eigen value = mean of sum of square distance
of 1st

(Variance)

Principal

Component //

PROCESS:

Dataset → Mean adjusted → Projected data on

(D)

PCs

• here all PCs may not
be imp

• Only (K) of them

Unit vector = $\bar{V}$

Projection of D on vector $\bar{V}$: $D \cdot \bar{V}$

Best fit vector $\bar{V}$

vector with maximum variance : $\max_{\bar{V}} \text{Var}(D \cdot \bar{V})$

NOTE: $\text{Var}(D \cdot V) = V^T S \cdot V$

S = covariance of matrix
of D

$$= \frac{1}{n-1} \sum_{i=1}^n (p_i - \bar{p})^2 \quad \text{Where } \bar{p} \text{ is projected sample mean}$$

$$= \frac{1}{n-1} \sum_{i=1}^n (V^T d_i - V^T \bar{d})^2 \quad \text{Where } \bar{d} \text{ is sample mean}$$

$$S = \frac{1}{n-1} \sum_{i=1}^n (d_i - \bar{d})(d_i - \bar{d})^T$$

NOTE:

Best vector that maximises the variance

$$\text{Var}(D \cdot v) = v^T S v$$

↑

$$\max_{v^T v = 1} v^T S v \quad \text{Since quadratic} \Rightarrow \text{no upper bound}$$

$$L(v, \lambda) = v^T S v - \lambda (v^T v - 1) \quad \text{maximise } (\lambda) = \text{lagrange multiplier}$$

$$\frac{\partial L}{\partial v} = 2 S v - 2 \lambda v = 0$$

$$S v = \lambda v \quad \leftarrow \text{eigen vector exp}$$

$$\lambda = v^T S v \quad \leftarrow \text{largest eigen vector} \\ \text{Principal eigen vector}$$

Also,

2nd PC $\perp$ 1st PC3rd PC $\perp$ 2nd PC, $\perp$ 1st PC alsoSo ~~the~~ nth PC should be $\perp$ to all 1 to (n-1) PC //

example. How to find PCA (or eigen vectors)

(i) compute mean

(ii) compute updated data points

$$(iii) \text{ Compute } S = \begin{bmatrix} \text{Cov}(u_1, u_1) & \text{Cov}(u_1, u_2) & \dots \\ \vdots & \vdots & \ddots \end{bmatrix} \quad (\text{determinant})$$

$$(iv) \text{ Solve: } |S - \lambda I| = 0 \quad \rightarrow \text{matrix}$$

$$(v) \text{ for max } \lambda : \text{ solve } (S - \lambda I) X = 0 \Rightarrow S X - \lambda X = 0$$

$$(vi) \text{ now make unit vector of } X \quad \lambda_i: \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

$$(vii) \text{ Compute: } X^T D \Rightarrow \text{for the projected points}$$

Day 13 (18/02/26)

→ not scalable

How to do matrix factorization?

- (i) Eigenvalue decomposition — for sq matrix
- (ii) Single Value " — for rectangular matrix

(i) eigenvalue decomposition

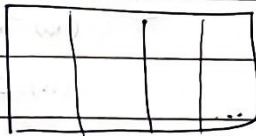
Matrix rank can be upto n

⇒ eigen vectors can be n (at most)

→ Eigen based PCA

- (i) Center data
- (ii) Compute covariance
- (iii) decompose
 $C = V \Lambda V^T$
- (iv) Take top k eigen vectors
- (v) Compress
 $Z = X V_k$

$$A \cdot V = \Delta \cdot V$$

$\lambda_1 \lambda_2 \lambda_3$


$\downarrow$

$\lambda_1 - PC1$
 $\lambda_2 - PC2$
 $\vdots$
 $\lambda_n - PCn$

$$A = V \Delta V^{-1}$$

for all A , V^{-1} may not exist

positive ⇒ V^{-1} exist

Semidefinite and if A^H will be orthogonal

$$= V^{-1} \text{ is } V^T$$

i.e V^{-1} orthogonal (vectors in V^{-1} orthogonal)

NOTE: positive definite
⇒ symmetric (square)
⇒ $V^{-1} = V^T$

⇒ here then columns in V vector gives P.C

NOTE:

$\begin{bmatrix} \lambda_1 & & & \\ & \lambda_2 & & \\ & & \lambda_3 & \\ & & & \lambda_k \\ & & & & 0 \\ & & & & & 0 \\ & & & & & & 0 \end{bmatrix}$

Low rank approximation

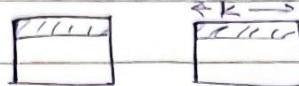
$\|A - A'\|$ should be minimum

→ here dimension reduced to k
⇒ in V , only first k columns imp
⇒ similarly for A

where, $A' = V_{n \times k} \Delta_{k \times k} V_{k \times n}^{-1}$

⇒ This is dimensionality reduction using eigen vectors
↳ (if V^{-1} exist then possible)

- now for reducing the dimension.

$$A_{n \times n} V_{n \times k} = A'_{n \times k}$$


- covariance matrix - symmetric

→ it includes eigen value decomposition
⇒ hence utilized in PCA technique

(ii) Singular value decomposition - (speech, text, Image and in)

→ Latent semantic indexing

(here the decomposition of data matrix happens and correspondingly it gives reduced form)

$$A V = \sigma U$$

right

↑ singular vectors

← left singular vector

→ a dirn such that

(Principal Components)

$$A \rightarrow \begin{matrix} \boxed{} \\ \end{matrix} \quad n \times m$$

$$\text{rank} \leq \min(n, m)$$

When the matrix acts on it, the output is just a scaled version of another orthogonal direction

$$\begin{matrix} A & V & = & U & \Sigma \\ \downarrow & \downarrow & & \downarrow & \downarrow \\ n \times m & m \times r & & n \times r & r \times r \end{matrix}$$

NOTE: V and U columns indicate

$$(U = A \cdot A^T)$$

= Eigen vectors of $A \cdot A^T$

$$A = U \Sigma V^T$$

~~the~~ ⁱth singular vector

$$\begin{pmatrix} V = A^T \cdot A \\ = \text{eigen vectors of } A^T \cdot A \end{pmatrix}$$

Σ : eigen values of $A \cdot A^T / A^T \cdot A$

$$A = U \Sigma V^T$$

$$A^T = (V \Sigma^T U^T)^T$$

classmate

Date _____

Page _____

NOTE :

$$\Sigma = \begin{bmatrix} \sigma_1 & & \\ & \sigma_2 & \\ & & \ddots \\ & & & \sigma_r \end{bmatrix}$$

$$\sigma_i^2 = \lambda_i \quad \text{--- (1)}$$

$$A \cdot A^T = U \Sigma \Sigma^T U^T$$

↘ Eigenvalue square of
singular vector

$$= U \Sigma (V^T V) \Sigma^T U^T$$

↘ Since V is orthogonal

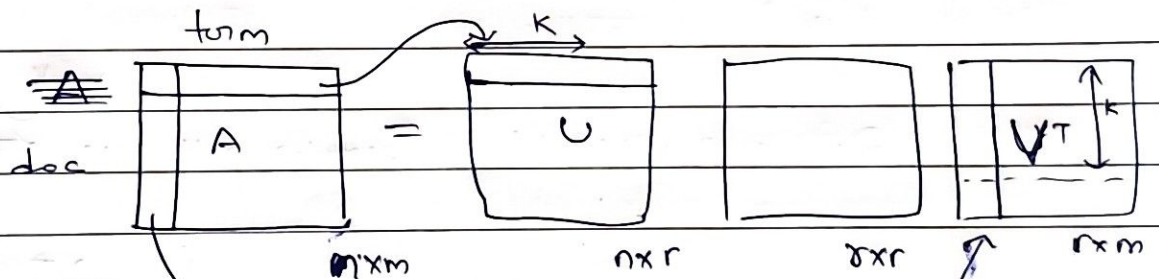
$$= U \Sigma \Sigma^T U^T$$

↘ eigen vectors of $A \cdot A^T$

(Relationship betn : Σ is singular vector of A

eigen vectors of $A A^T$ and singular vectors of A) $\Rightarrow$ (1) holds

$$\text{Similarly: } A^T A = V \Sigma \Sigma^T V^T$$



$$A' = U \Sigma' V^T$$

$n \times m \quad n \times k \quad k \times k \quad k \times m$

for a given,

$\Rightarrow$ each doc represented by V^T column
in reduced form (upto k rows)

$\Rightarrow$ Similarly, each term represented by U row
in reduced form (upto k columns)

d: What d' we get

in U?

$$d = d' \cdot \Sigma' \cdot V^T$$

$$\Rightarrow d' = \frac{1}{\| \Sigma' \|^2} \Sigma'^T V^T d$$

similarly for column use: U //

• Multilayer Perceptron

- req only bag of words representation
- no position considered



if position important $\Rightarrow$ NLP

(i.e context important)

• Recurrent Neural Network

↳ capture sequential info in the sentence

• CNN : image (1024x1024)

↳ dimension of vector huge : so to reduce dimension
localise features required to have to extract it

CNN was proposed



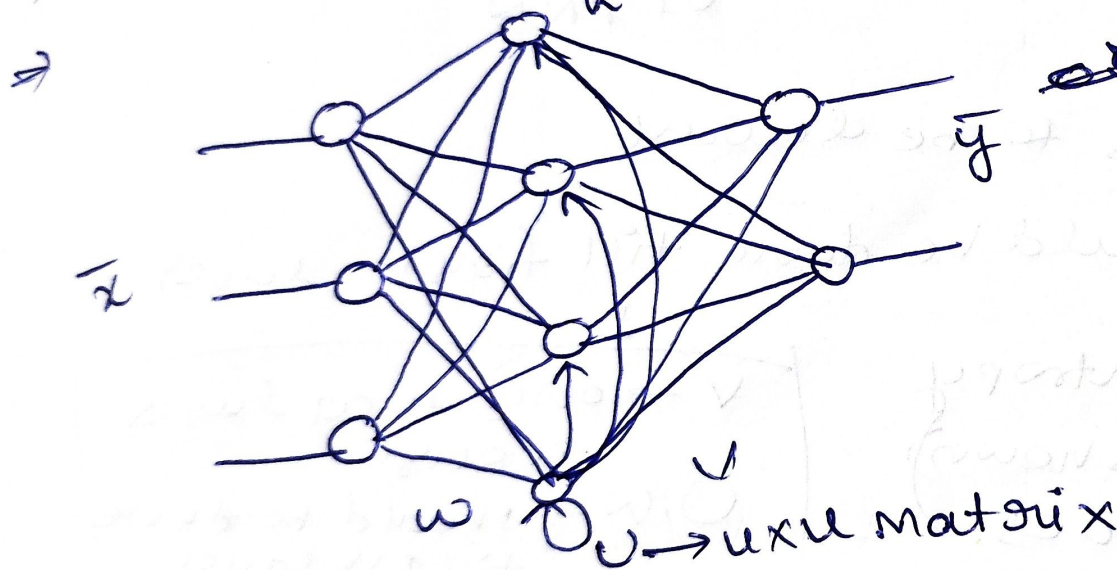
i.e; where MLP can't solve efficiently $\rightarrow$ we use RNN/CNN
 recurrent

$\rightarrow$ RNN $\rightarrow$ ~~order~~ order of features considered.

Image $\rightarrow$ problems $\rightarrow$ dimension large ~~and~~ (# parameters) & position of feature (localized features)

$\left. \begin{array}{l} \text{Solved} \\ \text{covered} \\ \text{in CNN.} \end{array} \right\}$

$\rightarrow$ recurrent neural network (RNN)



$\bar{h} = f(w^T \bar{x}) \rightarrow$ MLP

sometimes sequence of words is imp.
 for that applications we use this RNN.

for RNN, we have a feedback layer (internal connect - ions ~~on~~ b/w hidden layers neurons)

$$\bar{h}_1 = f(W^T \bar{x} + U^T \bar{h}_0)$$

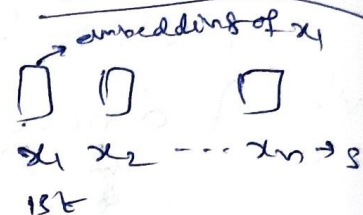
$\downarrow$ $\leftarrow$ timestamp (kind of) $\rightarrow$ neurons

which helps to take/consider sequentiality

previous ~~map~~ $\left\{ \begin{array}{l} \text{old takes} \\ \text{input to next calculations} \end{array} \right.$

$\downarrow$ For first evaluation, we can consider $\bar{h}_0 = \bar{0}$ (zero vector)

~~Previously sample is a vector~~
 Now sample is sequence of vectors



$$\bar{h}_2 = f(W^T \bar{x} + U^T (f(W^T \bar{x} + U^T \bar{h}_0)))$$

$\hookrightarrow$ as it considers both $h_0, h_1 \rightarrow$ we are able to capture

$\Rightarrow$ RNN $\rightarrow$ you can consider in diff ~~to solve~~ forms to solve different problems

$\left\{ \begin{array}{l} \text{due to self loop in hidden layer neurons} \\ \text{historical information} \end{array} \right.$

$\Rightarrow$ sequence of inputs to sequence of outputs $\rightarrow$ many to many
 (But the # inputs should be equal to # outputs)

$\Rightarrow$ many to one $\rightarrow$ last part ^(box) considered as classifier (dense layer)

$\Rightarrow$ sequence of inputs to sequence of outputs generated $\rightarrow$ last but one ^{box} ~~only~~ $\rightarrow$ many to many

$\Rightarrow$ we can do backward pass along with forward pass which can consider history as well as future at a point

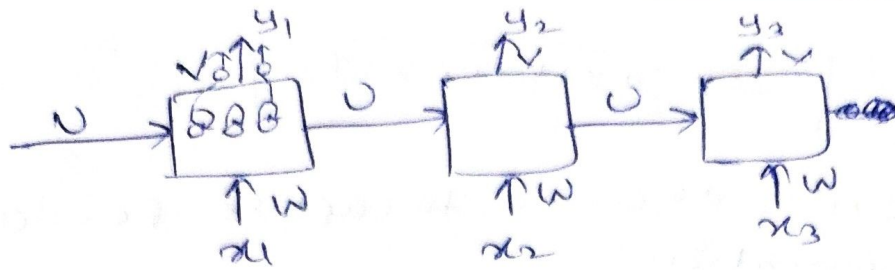
$\Rightarrow$ know which specific ~~config~~ configuration is suitable for our problem of RNN

$\Rightarrow$ $W, V, U \rightarrow$ parameters to be learnt

Backpropagation should be done till first layer

Error funn $\rightarrow$ cross entropy
 (In eg shown)
 in slides

$V \rightarrow$ only that layer enough
 $U, W \rightarrow$ should be done till 1st layer



$$\frac{\partial E^t}{\partial U} = \frac{\partial E^t}{\partial y^t} \times \frac{\partial y^t}{\partial z^t} \times \frac{\partial z^t}{\partial U}$$

recognition

$E(y)$

- If Many to one $\rightarrow$ calculating for one box is enough
- If many to many $\rightarrow$ should calculate for all boxes and do average / something else accordingly
- $t \Rightarrow$ Mean for all time stamps we need to calculate
- For which box/unit you start with, you have to backpropagate to first unit from that
- $\Rightarrow$ for every timestamp, backprop is expensive (for U, W)
- $\Rightarrow$ larger sequence $\rightarrow$ computationally expensive
- Multiple gradient multiplications $\rightarrow$ vanishing gradient problem (may become)
- $\Rightarrow$ same process for W as well (as U)
- $\Rightarrow$ To address vanishing gradient problem, Another Models are introduced
- ~~which~~ ~~are~~ ~~also~~ (LSTM, GRU)
- will discuss later.

- $\Rightarrow$ Convolutional Neural Network (CNN) $\rightarrow$ for text, Image, video, Etc. (used)
- $\Rightarrow$ How to vectorize the Image?
- concatenate all vectors (rows)
- $\Rightarrow$ Convolution $\rightarrow$ extraction (of localized features)
- pooling $\rightarrow$ summarization
- task-specific (dense layers $\rightarrow$ last network in Image slides)
- Deep CNN $\rightarrow$ convolution, pooling repeated Multiple times
- parameters of CNN $\rightarrow$ Filter (will be very small) (dense CNN)
- $\Rightarrow$ convolution :- (kernels)
- $\Rightarrow$ sum of element-wise Multiplications of filter, input
- $\Rightarrow$ stride $\rightarrow$ how much you are moving (position movement) of filter
- $\hookrightarrow$ Hyperparameter to set

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} * \begin{bmatrix} x & y \\ z & w \end{bmatrix} \rightarrow \boxed{ax+by+cz+dw}$$

⇒ padding → add 0 cells around to capture boundary information

⇒ we can have multiple filters (Each filter capturing diff aspects of Input Image) ^{Inherently} → multiple convolutions

⇒ Input matrix → can be Multi Dimensional

⇒ ~~padding~~ ~~padding~~

⇒ pooling :-

max of all elements in the Filter/matrix → max pooling

Arg pooling ⇒ Smoothing the Area } This is what ~~that~~ that inherently ~~does~~ does

Max pooling ⇒ Sharpening the area

⇒ CNN → sparsely connected MLP (Internally its MLP only instead of dense) MLP equivalent with CNN

⇒ u1 → 1st Node in hidden layers

Instead of all nodes → considers only q nodes to get value at ~~the~~ hidden layer

each connection is the filter

weighted summation of that particular wts and input ^{neurons}

* Filter says which ^{input} connections you're considering

Filter values in CNN

⇒ pooling → partitioning the connections

NOTE: $h^T = \sigma(\bar{x}^T W + b)$
 $= \sigma(\bar{x}^T W + \text{bias})$

class

• Multilayer Perceptron

- req only bag of words representation
- no position considered

Weights
associated
with bias
vector

↓
if position important $\Rightarrow$ NLP
(i.e context important)

• Recurrent Neural Network

↳ capture sequential info in the sentence

• CNN : Image (1024x1024)

↳ dimension of vector huge : so to reduce dimension
localise features required : hence to extract it

CNN was proposed

Day 14 (19/02/26)

→ machine translation system

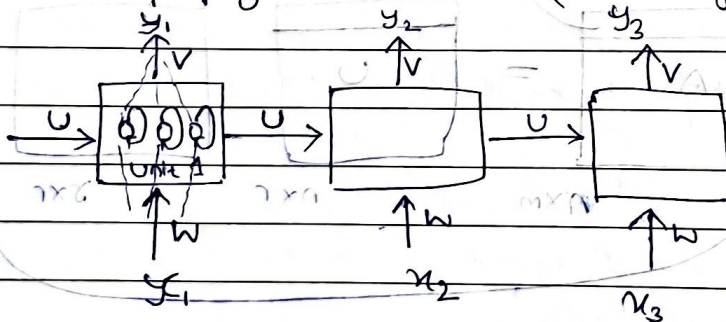
→ part of speech tagging

• RNN

→ What is it?

→ What are the parameters? W, U, V

• Backpropagation in RNN (also referred as through time)



(but depends on current timestamp only)

$$\frac{\partial E^t}{\partial v} = \frac{\partial E^t}{\partial y^t} \frac{\partial y^t}{\partial v}$$

$$\frac{\partial E^t}{\partial w} = \frac{\partial E^t}{\partial y^t} \frac{\partial y^t}{\partial w}$$

$$\frac{\partial E^t}{\partial u} = \frac{\partial E^t}{\partial h^t} \frac{\partial h^t}{\partial u}$$

perception

for every timestamp, we have to do! — if many to many

U : (right to left) — for u

- (i) forward
- (ii) backward
- (iii) bidirectional
- (iv) hybrid

RNN types: Many to many - ex. part of speech tagging
 Many to one - ex. guess the next word given an example sentence
 One to many - ex. tell as many words for an image
 One to one - ex. image classification

classmate
 Date _____
 Page _____
 Sentence _____

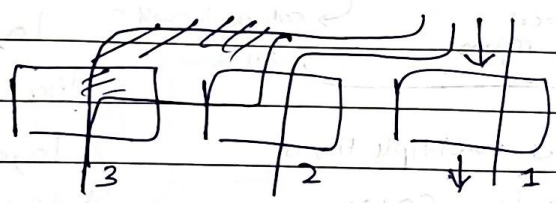
$$\frac{\partial E^t}{\partial U} = \frac{\partial E^t}{\partial y^t} \frac{\partial y^t}{\partial h^t} \frac{\partial h^t}{\partial U} + \frac{\partial E^t}{\partial y^t} \frac{\partial y^t}{\partial h^t} \frac{\partial h^t}{\partial h^{t-1}} \frac{\partial h^{t-1}}{\partial U}$$

+ ... till the last term in left

- no. of terms increases } large for large sequences } Computation
- lots of gradient multiplication
 ↳ vanishing gradient if deep network (as gradient ≈ 0)

$$= \frac{\partial E^t}{\partial y^t} \frac{\partial y^t}{\partial h^t} \frac{\partial h^t}{\partial h^{t-1}} \frac{\partial h^{t-1}}{\partial h^{t-2}} \dots \frac{\partial h^2}{\partial h^1} \frac{\partial h^1}{\partial U}$$

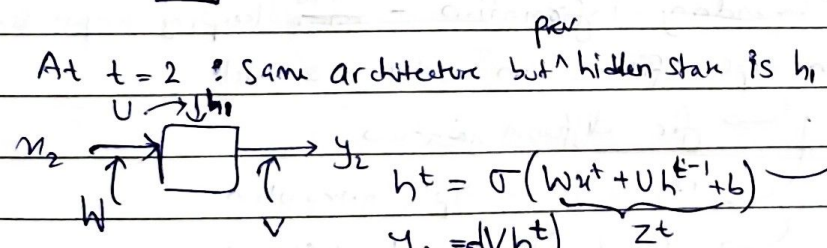
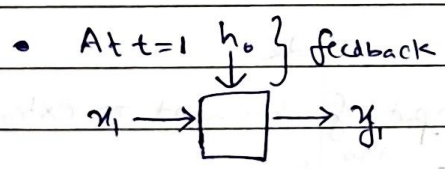
Similarly for $W \rightarrow$ we need to do like U



- for handling vanishing gradient pb $\rightarrow$ we have LSTM and GRU etc!

NOTE:

- In RNN $\rightarrow$ we use the same structure across time for different inputs i.e. W, U, V remains same



$$h^t = \sigma(Wx^t + U h^{t-1} + b)$$

$$y_2 = \sigma(V h^t)$$

or $V h^t \Rightarrow (h^t)^T V$

$h^t = \sigma(Wx^t + U h^{t-1} + b)$

W - input $\rightarrow$ hidden weights

U - hidden $\rightarrow$ hidden (recurrent weights)

$\downarrow$ carries memory across time

*NOTE: h^t is input seq till time t

• CNN

(i)



1024x1024x3



31,45,728

Why not MLP?

→ large no. of Params

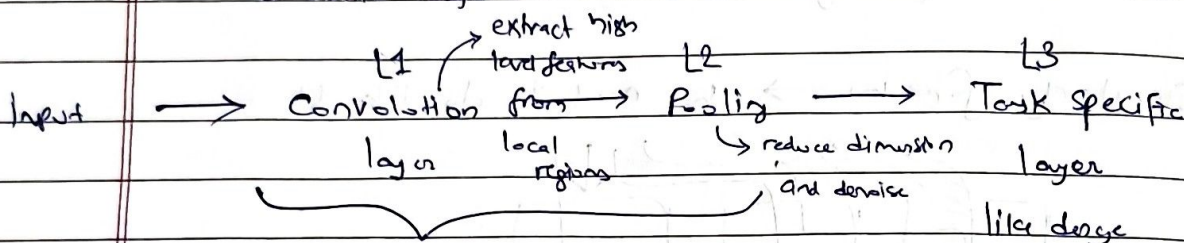
need of extracting high level local features
→ explicit features not specified

(ii) CNN — multilayer perceptron only

→ (but sparsely connected)

→ not dense connection

3 basic layer:



if repeated multiple times
⇒ deep CNN

design the networks for the underlying task

(iii) What is a local region?

→ filter: parameters for CNN

→ it tells what area we are focusing on

structural understanding of convolution

→ Convolution = input * filter



Size of convolution depends on stride

→ padding added on the input if we want to extract

boundary information — ~~also~~ keeping output size same

→ multiple filters can exist as well

→ for different features

→ results in multiple convolutions

→ size of filters can be different

→ input matrix can be multi dimensional as well

- reduces spatial size of convolved features
- decreases the computational power req to process the data
- extracts dominant features which are rotational and positional invariant //

(iv) Pooling

↳ after extracting the region, what to do?

↳ types: (i) max pooling → sharpening the area: (for age detection)

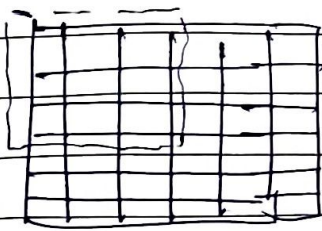
(ii) Average pooling → removing noise by removing non-dominant features
 ↑ Smoothing the values/area
 → remove noise by averaging

(v) Flattening of pooling layer



input to dense layer → decides what will be the work?

(vi) CNN - sparsely connected MLP



no dense connection

on applying filter, # sparse !!

→ 9 nos. selected from first 3 rows

36 values → all nos. not considered
 (9/36)

NOTE: same filter matrix applied !!

after convolution → we can have activation



pooling

↳ Flatten !!